LAKEHOUSEFIT AI

Complete Implementation-Ready PRD + Build Specification

hackCBS · IOMETE Track · Next.js · TypeScript · MongoDB

Purpose: Give a coding agent one precise specification to implement, test, and document a working end-to-end product.

Version 2.0 | Implementation specification

How to use this document

Copy the MASTER BUILD PROMPT in Appendix A into Antigravity (or another coding agent) after opening the project folder. Attach or paste this entire document as the source specification. Tell the agent to implement in the existing workspace, not merely describe code. The agent must inspect the workspace, create files, run commands, fix errors, and report exactly what was tested.

Important: no prompt can guarantee error-free code before execution. This specification makes implementation, testing, and honest reporting mandatory. Do not share production secrets in the prompt or commit .env files.

1. Product brief

1.1 Product

LakehouseFit AI is an evidence-backed buyer-readiness assessment tool for organizations evaluating whether documented IOMETE capabilities may address their business and technical requirements. It converts a guided assessment into requirement-level findings, source-backed explanations, unknowns, validation questions, next steps, and a downloadable PDF report.

1.2 Track alignment

The hackCBS IOMETE starter kit defines the mission as making complex technology easier to buy. It weights practical impact at 35%, buyer clarity at 25%, accuracy at 25%, and working execution at 15%. The product must solve one real buying workflow, show who it helps, how it works, and what it improves. It must not be a generic chatbot with a nicer front end.

1.3 Primary user

- Enterprise buyer / IT decision-maker who needs to understand whether IOMETE may fit a stated data-platform requirement.
- Secondary users: data engineering lead and sales representative preparing a discovery meeting.

1.4 Product promise

Understand your data challenges. Discover potentially relevant documented capabilities. Inspect the evidence. Identify what remains unknown. Know what to do next.

1.5 Non-goals

- Do not rebuild or run the IOMETE platform.
- Do not claim guaranteed compatibility, compliance, performance, ROI, pricing, or savings.
- Do not invent product features or mark claims verified just because an LLM sounds confident.
- Do not require an AI API for core assessment, matching, or PDF generation.
- Do not add unrelated dashboards, chat, accounts, or collaboration before the core journey works.

2. Judging strategy and proof

Criterion	Weight	Proof shown in live demo
Practical impact	35%	Complete an assessment and produce a reusable buyer-readiness report; compare against a defined manual baseline.
Buyer clarity	25%	Plain-English summary, role-aware

		explanations, clear statuses and next steps.
Accuracy	25%	Open a source for a claim; ask an unsupported question and show Unknown / Needs validation.
Working execution	15%	Live end-to-end path: form → matching → evidence → results → PDF download.

3. MVP scope and priorities

3.1 Must-have, release-blocking

- Responsive landing page with Start Assessment and Explore Example Report.
- Multi-step assessment wizard with validation, progress, back/next, and review-before-submit.
- Curated, source-backed IOMETE capability knowledge base.
- Deterministic, explainable requirement-to-capability matching engine.
- Requirement-level statuses: Supported, Needs validation, Unknown.
- Evidence cards with source title, URL, reviewed date/status, excerpt where permitted, and limitations.
- Results page with executive summary, matched capabilities, gaps, unresolved questions, and next steps.
- PDF report that mirrors the on-screen findings.
- Three or more fictional evaluation scenarios plus negative/unknown test cases.
- Unit tests for matching and evidence logic; end-to-end test for main workflow.
- Graceful operation without AI provider credentials.
- README, .env.example, setup instructions, build/test commands, and known limitations.

3.2 After MVP is stable

- Optional LLM-generated plain-English summaries grounded only in retrieved evidence.
- Optional saved assessments / report history with proper access control.
- Optional import of a buyer's questionnaire.
- Optional shareable report links, only if access control is implemented.

3.3 Explicitly defer

- Authentication, multi-tenant organizations, admin dashboards, web scraping at runtime, live company research, payments, and elaborate analytics unless time remains after acceptance tests pass.

4. Required screens and UI behavior

S1 — Landing page

- Clear headline: “Evaluate IOMETE with evidence, not guesswork.”
- Subheading explaining assessment → evidence → gaps → action plan.
- Primary CTA: Start assessment; secondary CTA: View sample report.
- Three-step process, example finding, limitations note, footer with source/methodology information.
- Sample report is clearly labelled illustrative, not an actual customer assessment.

S2 — Assessment wizard

- Visible step indicator and Save-in-session behavior; no data loss when moving backward.

- Step 1: role, industry, company size, assessment goal.
- Step 2: current infrastructure, data sources, analytics tools, approximate data volume (optional).
- Step 3: business challenges selected from a list plus optional free text.
- Step 4: deployment preference, data residency, access-control, integration, performance, availability and budget requirements (all optional unless essential to a chosen scenario).
- Step 5: review all inputs and edit before analysis.
- Clearly mark optional fields; never treat missing answers as requirements being satisfied.

S3 — Processing state

- Show truthful stages such as validating inputs, matching capabilities, checking evidence, preparing report.
- Do not fake percentages or claim a stage completed before it does.
- On failure, show retry and preserve the user's inputs.

S4 — Results dashboard

- Executive summary and a brief assessment scope.
- Finding cards grouped by Supported, Needs validation and Unknown.
- Each card shows requirement, why it matters, related capability, evidence link, limitations and next question.
- No unsupported overall “fit percentage” in the MVP.
- Actions: edit requirements, regenerate assessment, download PDF, start new assessment.

S5 — Evidence detail

- Show source title and URL, exact reviewed excerpt or concise evidence note, reviewed date, evidence status and limitations.
- If source cannot be verified, show that honestly; do not display a fabricated excerpt.

S6 — Report preview / download

- Preview the same structured findings shown on results screen.
- PDF includes scope, profile, executive summary, requirements, findings, evidence links, unknowns, risks, next steps, methodology and limitations.
- PDF must wrap text, paginate correctly and not clip long URLs or paragraphs.

S7 — Error / empty states

- Accessible error messages for validation, missing assessment, database outage, AI outage and PDF failure.
- Provide a recovery action and do not lose form state unnecessarily.

5. Functional requirements

FR-01: Assessment creation

Validate user inputs server-side; create a normalized assessment; return a stable assessment identifier; avoid accepting arbitrary client-generated findings.

FR-02: Requirement normalization

Convert structured choices and optional free text into requirement objects. Preserve original user text and let users review normalized requirements.

FR-03: Knowledge base

Store capabilities with IDs, descriptions, requirement tags, source title/URL, evidence state, review date and limitations. Seed only capabilities reviewed against official IOMETE documentation.

FR-04: Matching

Use deterministic rules and tags to find potentially relevant capabilities. A match means relevance only, not guaranteed implementation compatibility.

FR-05: Evidence

Every finding marked Supported must have a valid evidence record. Partially supported cases become Needs validation. No evidence becomes Unknown.

FR-06: Recommendations

Generate next steps from missing information and unresolved constraints using templates/rules. AI may improve phrasing but must not change evidence status.

FR-07: AI explanation (optional)

Use a server-only provider adapter; send only the minimum necessary data; ground output in supplied findings; validate response shape; handle timeout, invalid response, rate limits and missing key.

FR-08: PDF report

Generate report server-side from persisted or validated structured assessment results. Include source links, status definitions, generation date and disclaimers.

FR-09: Persistence

If MongoDB is configured, persist assessment and report records. If the DB is unavailable, show a clear error; do not falsely claim data was saved. A local demo mode may use in-memory/static scenarios, explicitly labelled.

FR-10: Sample scenarios

Provide at least three clearly labelled fictional sample cases and one unsupported-question case; keep sample records separate from real user assessments.

FR-11: Error handling

Use consistent error response shapes, safe user-facing messages and server logs without secrets or unnecessary personal data.

FR-12: Accessibility

Keyboard navigation, labels, focus indicators, semantic headings, readable contrast and status text not conveyed by colour alone.

6. Assessment statuses and matching policy

Supported

A documented capability is relevant to the stated requirement. This does not prove all customer-specific details or deployment compatibility.

Needs validation

Relevant documentation exists, but the exact customer requirement, integration, configuration or constraint needs confirmation.

Unknown

The available reviewed evidence is insufficient to make a reliable claim. Missing documentation does not prove the capability is absent.

MVP matching algorithm: normalize the requirement category; retrieve capability records tagged to that category; check evidence status and documented relationship; generate a finding with matching reason, evidence IDs, limitations and follow-up questions. Keep this algorithm deterministic and unit-testable. Do not calculate or display an arbitrary fit percentage.

7. Technical stack and implementation rules

Layer	Requirement
Framework	Next.js App Router, current stable version available in the environment, React, TypeScript.
Styling	Tailwind CSS and reusable accessible components (shadcn/ui may be used).
API	Next.js Route Handlers for server-side endpoints; no separate Express server required for MVP.
Validation	Zod schemas shared where appropriate, with mandatory server-side validation.
Database	MongoDB with Mongoose or official MongoDB driver. Use a reusable connection helper and indexes where needed.
Knowledge base	Start with reviewed JSON seed records; optionally sync to MongoDB. The core engine must work from a deterministic local seed.
AI	Optional provider adapter on server only. AI must not be a core dependency.
PDF	Server-side PDF generation using a maintained library, with pagination and link support.
Tests	Vitest for unit/integration tests; Playwright for main end-to-end flow.
Deployment	Vercel-compatible Next.js deployment and managed MongoDB, or another host that supports the chosen runtime and PDF library.

8. Suggested architecture and folder structure

```
src/
  app/
    page.tsx
    assessment/page.tsx
    assessment/results/[id]/page.tsx
    reports/[id]/page.tsx
    api/assessments/route.ts
    api/assessments/[id]/route.ts
    api/assessments/[id]/analyze/route.ts
    api/assessments/[id]/report/route.ts
    api/capabilities/route.ts
  components/
    layout/
```

```
    assessment/
    results/
    evidence/
    reports/
    ui/
  lib/
    db/connection.ts
    validation/
    matching/engine.ts
    matching/rules.ts
    evidence/
    ai/provider.ts
    ai/prompts.ts
    pdf/generate-report.ts
    errors/
  models/
    Assessment.ts
    Capability.ts
    Report.ts
  data/
    capabilities.json
    sample-scenarios.json
  types/
    assessment.ts
    capability.ts
    report.ts
tests/
  unit/
  integration/
  e2e/
.env.example
README.md
```

The coding agent may adapt the structure to the installed Next.js version, but must preserve separation between UI, validation, data access, matching, evidence, AI and PDF services.

9. Data contracts

9.1 Capability record

```
{
  "id": "deployment-self-managed",
  "name": "Self-managed deployment",
  "category": "deployment",
  "description": "Plain-language description reviewed against official documentation.",
  "relevantRequirementTags": ["infrastructure-control", "deployment-choice"],
  "source": {
    "title": "Verified official page title",
    "url": "https://official-source-verified-during-research",
    "excerpt": "Short reviewed excerpt or evidence note",
    "verifiedAt": "ISO-8601 date"
  },
  "evidenceStatus": "documented",
  "limitations": ["Customer-specific deployment needs validation."]
}
```

The URL above is a schema placeholder, not a verified citation. The implementation agent must replace it with actual official IOMETE URLs after reviewing the documentation. Never seed fabricated citations.

9.2 Assessment record

```
{
  "organization": {
    "industry": "retail",
    "companySize": "medium",
    "userRole": "it_decision_maker",
    "companyName": null
  },
  "environment": {
    "deployment": "own_cloud",
    "dataSources": ["sales systems"],
    "analyticsTools": [],
    "dataVolume": null
  },
  "requirements": [
    {
      "id": "req-001",
      "category": "deployment",
      "description": "Keep analytical data in a controlled environment",
      "priority": "high",
      "source": "structured_input"
    }
  ],
  "status": "completed"
}
```

9.3 Finding record

```
{
  "requirementId": "req-001",
  "status": "needs_validation",
  "relevantCapabilityIds": ["deployment-self-managed"],
  "explanation": "The documented deployment option may be relevant to this requirement.",
  "evidenceReferences": ["deployment-self-managed"],
  "unresolvedQuestions": ["Confirm the exact infrastructure and isolation requirements."],
  "limitations": ["No customer-specific compatibility determination has been made."]
}
```

10. API contracts

Endpoint	Expected behavior
POST /api/assessments	Validate and create an assessment; return ID and status.
GET /api/assessments/:id	Return assessment and result if available; enforce access rules for persisted private data.
POST /api/assessments/:id/analyze	Run deterministic matching and evidence evaluation; idempotent where feasible.
GET /api/capabilities	Return curated capability summaries and source references.
GET /api/assessments/:id/report	Generate/download PDF from validated structured results.

All endpoints must have typed input/output contracts, server-side validation, appropriate HTTP status codes, safe error messages and tests. If anonymous assessments are supported for the demo, use a safe access design and avoid exposing sensitive information through guessable IDs.

11. Environment configuration

```
# Required only if MongoDB persistence is enabled
MONGODB_URI=

# Optional: configure only the provider actually implemented
AI_PROVIDER=
AI_API_KEY=
AI_MODEL=

# Optional public app origin, if required by the chosen deployment
NEXT_PUBLIC_APP_URL=
```

The agent must document which variables are actually required. Never put AI_API_KEY or MONGODB_URI in a NEXT_PUBLIC_ variable. Provide .env.example with empty placeholders; never invent credentials.

12. Security and privacy requirements

- Validate every API input on the server and cap request sizes.
- Keep database credentials and AI keys server-side only.
- Do not log secrets, full API keys or unnecessary sensitive free-text.
- Sanitize rendered user text and guard against XSS.
- Use safe report access controls if data is persisted or shared.
- Do not claim a report is private, deleted or encrypted unless the implementation actually provides that property.
- Use a restrictive CORS policy only if cross-origin APIs are introduced; same-origin Next.js routes should remain same-origin.
- Handle prompt injection in free-text as untrusted input; it must never override system rules or evidence statuses.
- Do not send unnecessary identifying or confidential company data to an AI provider.
- Document retention behavior and any limitations of anonymous demo storage.

13. Test plan and acceptance gates

13.1 Required scenarios

- Scenario A: fictional organization with infrastructure-control requirements; show potentially relevant documented capabilities and validation questions.
- Scenario B: fictional retailer focused on SQL analytics and growing workloads; match only capabilities actually documented in the knowledge base.
- Scenario C: organization with incomplete requirements; ask follow-up questions and avoid an overconfident conclusion.
- Scenario D: unsupported certification/compliance question; return Unknown and explain what evidence is needed.
- Scenario E: conflicting or malformed input; validate and recover without crashing.
- Scenario F: AI provider missing or unavailable; complete assessment and PDF using deterministic output.

13.2 Automated tests

- Unit tests for each matching status and evidence rule.
- Unit tests that prevent Unknown or partially documented capabilities from being labelled Supported.
- API tests for invalid input, missing records and server errors.
- PDF test confirming report contains all key sections and does not fail on long text.

- Playwright test for landing → assessment → results → evidence → PDF download.
- Build check and lint/type check using the actual scripts in package.json.

13.3 Definition of done

- Production build succeeds.
- Lint and type checking pass, if configured.
- All critical unit/integration/E2E tests pass, or failures are clearly reported with causes.
- No fake evidence links remain in the knowledge base.
- Every finding shown as documented has a reviewed official source.
- Core workflow works without an AI API key.
- PDF download works.
- Responsive layout and keyboard basics are checked.
- README explains setup, environment variables, seed data, tests and deployment.
- Known limitations are documented honestly.

14. Demo script (5 minutes)

1. 0:00–0:45 — Explain the buyer problem and the cost of unstructured research; do not claim unmeasured savings.
2. 0:45–1:45 — Complete the assessment for a fictional organization.
3. 1:45–2:45 — Show findings and open the source supporting one capability.
4. 2:45–3:30 — Ask an unsupported question and show Unknown / Needs validation.
5. 3:30–4:20 — Download and inspect the report.
6. 4:20–5:00 — Show real test results, limitations and how the report helps the next buying step.

15. Risks and mitigations

Hallucinated product claims

Only verified capability records may support a documented claim; unknowns remain explicit.

API availability

Core workflow and PDF use deterministic logic; AI is optional.

Scope creep

Freeze MVP scope until end-to-end acceptance gates pass.

Weak or stale sources

Record source title, URL and review date; manually verify sources before demo.

Privacy exposure

Minimize data collection and implement appropriate report access.

Looks complete but is not functional

Require working API/database paths, tests and a real production build; no completion claim without evidence.

16. Deliverables the coding agent must leave in the workspace

- Complete Next.js source code.
- Responsive UI and all required screens.
- Functional assessment wizard and results workflow.
- Reviewed capability knowledge base with actual official sources.
- Working deterministic matching engine.
- Optional AI provider adapter with fallback.
- Working PDF generation and download.
- MongoDB persistence if configured, with clear behavior when unavailable.
- Unit/integration/end-to-end tests.
- .env.example with placeholders only.
- README with exact installation, run, test, build and deployment commands.
- A short IMPLEMENTATION_STATUS.md listing completed features, test commands and actual results, unresolved issues and manual setup still required.

Appendix A — Master build prompt (copy and paste into Antigravity)

```
You are the senior full-stack engineer responsible for IMPLEMENTING LakehouseFit AI from the attached
complete PRD. Work directly in the current project workspace. Do not respond with a plan only. Inspect the
workspace, implement the application, run it, test it, fix errors and document the actual result.

SOURCE OF TRUTH
- Follow the attached PRD and the IOMETE hackCBS starter-kit mission: make complex technology easier to
buy.
- Optimize for practical impact (35%), buyer clarity (25%), accuracy (25%) and working execution (15%).
- Build one coherent end-to-end buyer assessment workflow, not a generic chatbot.
- Never invent IOMETE capabilities, certifications, pricing, benchmarks, compatibility or savings.
- Verify all product claims against actual official IOMETE documentation. Replace every placeholder URL
with a real, reviewed source. If a claim cannot be verified, label it Unknown and do not present it as
fact.

TECHNOLOGY
- Use Next.js App Router, React, TypeScript and Tailwind CSS.
- Use Next.js Route Handlers for server-side APIs.
- Use Zod for validation and MongoDB with Mongoose or the official driver for persistence.
- Use a deterministic, unit-testable matching engine.
- AI is optional and server-side only; the core workflow must work without an AI key.
- Implement a working PDF report download.
- Use Vitest and Playwright (or equivalent compatible tests) for unit and end-to-end validation.

REQUIRED END-TO-END FEATURES
1. Landing page with Start Assessment and an illustrative sample report.
2. Multi-step assessment wizard: organization, data environment, challenges, technical requirements,
review.
3. Form validation, progress, back/next, preserved inputs and accessible errors.
4. Structured requirements and a reviewed capability knowledge base with source title, actual official URL,
evidence status, review date and limitations.
5. Deterministic matching that generates Supported, Needs validation or Unknown findings. These statuses
refer to evidence/relevance and never guarantee customer-specific compatibility.
6. Results dashboard with executive summary, capability findings, evidence links, unknowns, unresolved
questions and next steps.
```

```
7. Evidence details view with source and limitations.
8. PDF report generated from the same structured results shown on screen.
9. Three or more fictional scenarios and explicit unsupported-question test cases.
10. Optional AI explanations grounded only in supplied findings; handle missing keys, timeouts, invalid output and provider errors.
11. MongoDB persistence when configured. Never claim persistence succeeded if the database failed. If a demo fallback exists, label it clearly.
12. Useful empty, loading, success and error states.
13. Responsive desktop/mobile layout with keyboard accessibility.
14. README, .env.example and IMPLEMENTATION_STATUS.md.

IMPLEMENTATION RULES
- First inspect the existing files and package manager; preserve useful existing work.
- If the workspace is empty, scaffold a complete Next.js TypeScript project.
- Implement in small stages, but continue through the whole PRD rather than stopping after the plan or landing page.
- Do not leave TODOs, fake buttons, dead links, placeholder API responses or nonfunctional primary actions in the required workflow.
- Sample data is allowed only when clearly labelled as fictional. Keep it separate from real assessments.
- Do not put secrets in client code or commit them. Never invent credentials.
- Validate every server-side request. Use safe error messages.
- Use a consistent premium enterprise SaaS design: clean white/neutral backgrounds, deep green accent, strong typography, clear spacing, accessible contrast and subtle borders/shadows. Prioritize clarity over decoration.
- Do not add features outside the MVP until the acceptance tests pass.

EXECUTION AND VERIFICATION
1. Inspect the workspace and identify available runtime/tooling.
2. Implement all required screens, data models, API routes and services.
3. Seed the knowledge base only with capabilities whose official sources have been reviewed. Keep an audit-friendly source field.
4. Write unit tests for matching and evidence status rules, API/integration tests, and an end-to-end test for the main journey and PDF download.
5. Install dependencies using the existing package manager; do not casually replace the lockfile.
6. Run lint, type check, unit/integration tests, production build and E2E tests where the environment supports them.
7. When a command fails, inspect the error, fix the root cause and rerun it. Do not hide failures or claim tests passed without running them.
8. Manually verify the user journey at desktop and mobile sizes if browser tools are available.
9. If a service needs credentials or external setup that you cannot complete, implement safe configuration handling, document the exact remaining step and continue verifying everything else.
10. Do not stop at a plan. Keep working until the required workflow is implemented or a genuine blocker is reached.

FINAL RESPONSE REQUIRED
- Summarize what was implemented.
- Give exact commands to install, run, test and build.
- List actual commands run and their pass/fail results.
- List any missing credentials or manual setup.
- List known limitations and incomplete requirements.
- Link to the relevant files if supported.
- Never claim the project is fully working unless the actual verification supports that claim.
```

Appendix B — Recommended first run commands

```
npm install
npm run dev

# Before declaring the project complete, run the scripts actually defined in package.json:
npm run lint
```

```
npx tsc --noEmit
npm test
npm run build
npx playwright test
```

The coding agent must check package.json and adjust commands to the scripts and test tools actually installed. Do not report a command as passed unless it was executed successfully.

Appendix C — Hackathon evidence checklist

- The problem and target buyer are stated in one sentence.
- The live workflow completes from assessment to PDF.
- At least one product finding opens a real official source.
- At least one unsupported question returns Unknown.
- The demo uses clearly labelled fictional company data.
- The test set and actual results are visible.
- No invented ROI, savings, compliance or performance claim is presented.
- The report explains next steps and limitations.
- The application remains usable if the AI provider is unavailable.
- The team can explain the matching logic and data flow.